

What Determines the Direction a Bone is Pointing in 3D Animation?

In 3D animation systems (like FBX from Blender), a bone's "direction" refers to the orientation of its local axis—essentially, where it "points" in space relative to its parent or the world. This is determined by a combination of position, rotation, and hierarchy. I'll break it down step by step, focusing on mathematical concepts and Blender/FBX specifics, as per your code context. I'll avoid relying on anecdotal posts and stick to core principles from documentation (e.g., Blender manual, FBX spec) and math.

1. Basic Bone Structure and Direction

- A bone is defined by two points: **head** (start position) and **tail** (end position).
- The **direction vector** is simply:

$$d = tail - head$$

$$\text{Normalized: } \hat{d} = \frac{d}{||d||}$$

This gives the bone's "pointing" direction along its length. In code (like your Bone.cs), this isn't explicitly stored but implied by LocalRest transform (which includes translation from parent).

- In Blender (your export source), bones point along the **+Y axis in local space** by default. The tail is at (0, length, 0) relative to head at (0,0,0). This is a convention: Y is "forward" for bones, while Z is up for the scene.
- **Initialization Issue Potential:** If parsing (e.g., in FBXSkeletonParser.cs) doesn't correctly remap axes from Blender's Y-up to your engine's Z-up, bones might initialize pointing in the wrong direction (e.g., along Z instead of Y), causing XY plane mismatches. Your diff shows identity remapping, which preserves Blender's Y-pointing but may conflict with engine expectations.

2. Role of Rotations (Euler Angles) in Direction

- Bones don't "translate" with Eulers—Euler angles are for **rotation**, which orients the bone's direction relative to its parent.
- **Euler Rotation Basics:** Eulers represent rotation as three sequential angles (X, Y, Z) around axes. The order (e.g., XYZ in your Bone.cs) matters:
 - Rotation matrix = $R_z(\theta_z) * R_y(\theta_y) * R_x(\theta_x)$ (your code's composition).
 - The final direction is the initial vector (e.g., +Y) transformed by this matrix.
- **Direction of Euler "Translation":** I think you mean "Euler rotation direction." Eulers don't translate; translation is separate (LclTranslation in your code). But rotations determine pointing:
 - Positive angles are **counter-clockwise** (right-hand rule) around the axis.
 - For a bone pointing +Y initially:
 - +X rotation tilts it left/right in XY plane.
 - +Y rotation pitches up/down.
 - +Z rotation rolls it.
 - In Blender/FBX, positive is counter-clockwise; negative is clockwise. This is standard (confirmed in Blender docs).
- **Per-Bone Issues in XY Plane:** If adjacent bones have opposite rolls (common in symmetric rigs, like left/right arms with $\pm 90^\circ$ roll for mirroring), their directions offset by $90^\circ/180^\circ$ in XY. Your symptoms match this—initialization fails if parsing doesn't handle roll correctly (e.g., no inversion of PostRotation as in my previous fix, which helped but didn't fully resolve relations).

3. Hierarchy and Parent-Child Relationships

- Bones are hierarchical: A child bone's direction is relative to its parent's local space.
- Global direction = Parent's global transform * Child's local transform * Direction vector.
- In your code (Skeleton.ComputeGlobalTransforms): `global[i] = global[parent] * local[i]`.
- **Initialization Issue:** If LocalRest (from ComputeLocal) is wrong due to unhandled pre/post rotations or geometric transforms, child bones misalign with parents. Your diff inverted bind poses but "moved the issue"—likely because cluster transforms in FBXMeshParser (tl_remap, t_remap) need similar handling for skinning, ensuring vertex positions align with bone directions.

4. How to Know for Sure (Verification Methods)

- **Mathematically:** Compute the bone's local Y-axis after rotation: Transform `Vector3.UnitY` by the rotation matrix from `CreateFromEuler`. If it doesn't match expected direction (e.g., tail - head normalized), initialization is off.
- **In Blender:** Enable "Axes" display in armature properties. Bones point +Y (green). Export FBX, import back— if directions match, issue is in your parser.
- **In Code/Debug:** Log global tail positions (`head + length * transformed UnitY`). Compare to expected (e.g., in viewer, check if adjacent bones align along chain without 90° kinks).
- **FBX Spec Confirmation:** FBX bones use local transforms with pre/post rotations for compatibility (e.g., Maya/Blender diffs). Direction is along local Y by convention, but customizable via roll.
- **Test:** Add logging in `ComputeLocal` to output the final local matrix's forward vector (`M21, M22, M23` for row-major). It should be close to (0,1,0) for default bones.

5. Potential Initialization Fix in Your Code

- The issue is likely in how local transforms compose without accounting for Blender's roll in post-rotation. Previous `inv(Post)` helped but not fully—try also inverting Pre or swapping order to `Post * R * Pre` (math shows this reorients relations).
- Next step: Request `FBXParserUtils.cs` for value interpolation checks, as animation curves might misapply Eulers.

If this clarifies, what's the specific bone pair with offset (e.g., arm_l and arm_r)? Or provide logs from ToEuler.